

BISSPA NAVTEX

Decodificador NAVTEX para Android
SITOR-B / CCIR-476 · FSK 100 Bd (1615/1785 Hz)

Código fuente completo del proyecto compilable
documentación técnica, instrucciones de compilación y uso

Versión 1.0 · septiembre de 2026
Paquete: com.radiusb1.radio · minSdk 26 (Android 8.0)

1. Descripción de la aplicación

BISSPA NAVTEX es una aplicación Android que recibe, a través del micrófono del teléfono, los tonos FSK de una emisora NAVTEX y muestra los mensajes de navegación y meteorología en un terminal estilo consola.

NAVTEX (Narrow Band Direct Printing, NBDP) transmite a 100 baudios con desplazamiento de frecuencia: 1615 Hz = mark (bit 1) y 1785 Hz = space (bit 0). La codificación de caracteres es el código CCIR-476 (Baudot/ITA-2 de 7 bits con corrección de errores) y la capa de enlace es SITOR-B (ARQ con diversidad de tiempo): cada carácter se transmite dos veces separadas 5 ranuras (≈ 280 ms), lo que permite corregir errores de transmisión.

La app implementa el receptor completo: demodulación Goertzel, decodificador de bits con corrección de sincronismo de carácter, detección de la alineación DX/RX (phasing α/β o estimador estadístico), FEC comparando las dos copias, cambios letras/figuras y el formato de mensaje NAVTEX (ZCZC + letra B de estación y mensaje + NNNN).

Funciones principales

- Demodulación FSK en tiempo real desde AudioRecord (micrófono, 48 kHz).
- Detección de señal (promediado Goertzel) y nivel de audio en pantalla.
- Corrección de errores SITOR-B (FEC, borrados cuando no hay solución) y auto-resincronización.
- Terminal con colores: mensajes en verde, estados en ámbar/azul.
- Detección de ZCZC/NNNN con ventana de encuadre y avisos en pantalla.
- Operación continua estable (bloqueo de acceso, límite de memoria del terminal).

2. Contenido del proyecto

Estructura de carpetas del proyecto Android Studio (BisspaNavtex/):

```
BisspaNavtex/  
├─ settings.gradle.kts  
├─ build.gradle.kts  
├─ gradle.properties  
├─ gradle/wrapper/gradle-wrapper.properties  
└─ app/  
    ├─ build.gradle.kts  
    ├─ proguard-rules.pro  
    └─ src/main/  
        ├─ AndroidManifest.xml  
        ├─ java/com/radiusb1/radio/  
        │   ├─ MainActivity.kt          (pantalla principal)  
        │   └─ FSKActivity.kt        (decodificador NAVTEX)  
        └─ res/  
            ├─ layout/activity_main.xml  
            ├─ layout/activity_fsk.xml  
            ├─ values/strings.xml  
            ├─ values/colors.xml  
            ├─ values/themes.xml  
            ├─ drawable/ic_launcher_foreground.xml  
            └─ mipmap-anydpi-v26/ic_launcher.xml
```

3. Requisitos y compilación

Requisitos:

- Android Studio Hedgehog (2023.1) o posterior, o bien JDK 17 + Android SDK 34 por línea de

comandos.

- Teléfono con Android 8.0 (API 26) o superior, con micrófono.
- Gradle 8.4 · Android Gradle Plugin 8.2.2 · Kotlin 1.9.22 (ya configurados en el proyecto).
- Solo se usan librerías de AndroidX (appcompat y core-ktx); no hay más dependencias.

Compilación con Android Studio:

1. File → Open... y selecciona la carpeta BisspaNavtex (la que contiene settings.gradle.kts).
2. Espera la sincronización de Gradle (descarga AGP/Kotlin/AndroidX la primera vez que haya Internet).
3. Conecta el teléfono con depuración USB activada.
4. Pulsa Run ► y elige el dispositivo: se instala y se abre la app.

Compilación por terminal:

```
cd BisspaNavtex
./gradlew assembleDebug          # genera app/build/outputs/apk/debug/app-debug.apk
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Nota sobre el wrapper: el proyecto incluye gradle/wrapper/gradle-wrapper.properties (Gradle 8.4). Al abrirlo en Android Studio se usa automáticamente. Si se compila por terminal y no existiera el script gradlew, basta ejecutar una vez «gradle wrapper» (o copiar gradle/wrapper/gradle-wrapper.jar desde cualquier proyecto con Gradle 8.4).

4. Uso

1. Sintoniza una emisora NAVTEX en una radio en USB (banda lateral superior):
 - 518 kHz inglés, servicio internacional
 - 490 kHz servicio local (por ejemplo, mensajes en español)
 - 4209,5 kHz servicio HFEn radios sin sintonía exacta: sintoniza ~1,7 kHz por encima y usa USB (la portadora va ~1,7 kHz por debajo de la frecuencia marcada).
2. Coloca el teléfono cerca del altavoz de la radio (o conecta la salida de auriculares con un cable). El indicador «Nivel Audio» confirma la recepción.
3. Abre «Abrir decodificador NAVTEX» y deja la pantalla encendida.
4. El terminal imprime los mensajes al detectar ZCZC (inicio) ... NNNN (fin).

Consejos: volumen medio-alto sin saturar (si el nivel supera 100 % de forma continua, bájalo); sin música ni ruido de fondo; el tono de dos frecuencias debe oírse limpio. En la terminal, el guion bajo «_» indica un carácter borrado (error no corregible) y los cambios de figuras/cifras («+», «-») el estado del código Baudot.

5. Notas técnicas del decodificador

Demodulación: filtro Goertzel de dos tonos (1615 Hz mark / 1785 Hz space) sobre bloques de 20 muestras por símbolo (48 kHz / 100 Bd = 480 muestras por símbolo, ventanas de 20). Un clasificador bayesiano por bit (log-verosimilitud de los 20 sub-bloques) tolera distorsión de fase y de amplitud. En los cruces de cero se valida y recalibra la fase de muestreo de símbolo.

Framing CCIR-476: los 35 símbolos válidos tienen peso Hamming 4 en 7 bits y se buscan como el patrón 01111110 (estándar de 7 bits + un bit de paridad que lo completa). Un bit de calidad baja produce un símbolo borrado sin romper el framing. Si se acumulan demasiados bits débiles o el anillo de audio se desborda, el reloj re-alinea buscando la mayor racha de grupos válidos entre 7 desfases de carácter.

FEC SITOR-B: cada carácter viaja dos veces, una en «posición DX» (información nueva) y otra en «posición RX» (copia de la que viajó 5 ranuras antes $\approx$ 280 ms de diversidad de tiempo). En posición DX se emite el carácter nuevo; en RX se emite solo si se recibió borrado la primera vez o si ambas copias son iguales; con conflicto se emite borrado «_» (o la copia RX si era sana). La alineación DX/RX se detecta con los patrones de phasing (α en posiciones DX, β en posiciones RX, racha mínima de 3 alternancias) o, si no hay phasing, con un estimador estadístico de aciertos/fallos por posición.

Alfabeto: tabla ITA-2 internacional (CCIR 476-2): S=apóstrofe, J=BEL (omito), D=ENQ, H=libra «£». La variante americana US-TTY (S=\$, J=#, D=\$ etc.) solo se comenta en el código. El bit 7 es la paridad constante 1 (se invierte al calcular el patrón 7-bit).

Formato NAVTEX (Rec. ITU-R M.541-10): ZCZC « $\perp$ » B1C00 ... mensaje ... NNNN. La letra B al principio (B-Y) identifica la estación emisora y la cifra siguiente (1-9) el tipo de aviso (A=navegación, B=meteorología...). Los cambios letras/figuras del Baudot se restablecen al empezar cada mensaje.

Correcciones respecto del código original

1. Búsqueda de alineación de carácter (7 desfases; el original no la tenía).
2. Bits débiles $\rightarrow$ símbolo borrado sin perder el framing de 7 bits.
3. Re-sincronización del reloj cuando el anillo de audio se sobrescribe (antes: bloqueo silencioso).
4. Seguridad de hilos (decoderLock + campos @Volatile para audio/UI).
5. Tabla de figuras ITA-2 internacional correcta (' , £, BEL, ENQ...).
6. Puerta FEC corregida: en posición DX se emite el carácter nuevo sin compararlo con la copia de 5 ranuras atrás (el original la comparaba y emitía duplicados o basura).
7. figuresMode se restablece al empezar mensaje (ZCZC).
8. NNNN se imprime una sola vez (el original lo hacía dos).
9. CR/LF entran en la ventana de encuadre ZCZC/NNNN (antes falseaban NNNN prematuramente).
10. Latch de phasing por racha de alternancias $\alpha/\beta \geq 3$ (antes: primer par, con ruido se desenganchaba).
11. Decaimiento de la evidencia eliminado (revolvía el modo de paridad).
12. Optimizaciones: clasificación de bit una sola vez por fase (reutilizada en ambas polaridades y las 7 alineaciones), Goertzel con coeficientes precalculados, terminal por lotes de 150 ms con límite de 20 000 caracteres, System.arraycopy del anillo, tabla CharArray(128) en vez de HashMap, reintentos de adquisición solo con $\geq$ 250 ms de audio nuevo.

Validación (simulador de canal, mensajes de 320 caracteres)

Errores de bit aleatorios; se mide la distancia de edición media por mensaje y el porcentaje de

mensajes con detección correcta de ZCZC:

p error	edición (orig)	edición (nuevo)	ZCZC orig	ZCZC nuevo

0.000	0.00 (4/4 exactos)	0.00 (4/4 exactos)	100 %	100 %
0.005	16.33	0.15	84 %	100 %
0.010	22.79	1.94	74 %	97 %
0.020	31.59	7.23	61 %	89 %
0.040	42.28	24.73	43 %	64 %

Sin phasing (fallback estadístico, 320 caracteres): p=0 → 4/4 exactos; p=0.01 → 1 carácter perdido;
p=0.02 → 12/320 perdidos.

6. Código fuente completo

A continuación se listan los 17 ficheros del proyecto en el mismo orden de la estructura del apartado 2. El contenido es exactamente el del proyecto BisspaNavtex (copia íntegra, listo para pegar en Android Studio).

settings.gradle.kts (17 líneas)

```
pluginManagement {
    repositories {
        google()
        mavenCentral()
        gradlePluginPortal()
    }
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "BisspaNavtex"
include(":app")
```

build.gradle.kts (5 líneas)

```
// Fichero de build raíz (versiones de los plugins)
plugins {
    id("com.android.application") version "8.2.2" apply false
    id("org.jetbrains.kotlin.android") version "1.9.22" apply false
}
```

gradle.properties (4 líneas)

```
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
android.useAndroidX=true
android.nonTransitiveRClass=true
kotlin.code.style=official
```

gradle/wrapper/gradle-wrapper.properties (5 líneas)

```
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionUrl=https\://services.gradle.org/distributions/gradle-8.4-bin.zip
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

app/build.gradle.kts (41 líneas)

```
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.radiusb1.radio"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.radiusb1.radio"
        minSdk = 26
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }
}
```

```

compileOptions {
    sourceCompatibility = JavaVersion.VERSION_17
    targetCompatibility = JavaVersion.VERSION_17
}

kotlinOptions {
    jvmTarget = "17"
}
}

dependencies {
    implementation("androidx.core:core-ktx:1.12.0")
    implementation("androidx.appcompat:appcompat:1.6.1")
}

```

app/proguard-rules.pro (2 líneas)

```

# BISSPA NAVTEX – reglas ProGuard/R8 (la app no usa minify)
# No se necesitan reglas adicionales.

```

app/src/main/AndroidManifest.xml (34 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Decodificación de audio: lectura del micrófono -->
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-feature
        android:name="android.hardware.microphone"
        android:required="true" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:supportRtl="true"
        android:theme="@style/Theme.BisspaNavtex">

        <!-- Pantalla principal -->
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Decodificador NAVTEX -->
        <activity
            android:name=".FSKActivity"
            android:exported="false"
            android:screenOrientation="portrait" />

    </application>
</manifest>

```

app/src/main/java/com/radiusb1/radio/MainActivity.kt (22 líneas)

```

package com.radiusb1.radio

import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

/**
 * Pantalla principal de BISSPA NAVTEX.
 * Abre el decodificador NAVTEX (FSKActivity).
 */
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        findViewById<Button>(R.id.btnAbrirFSK).setOnClickListener {
            startActivity(Intent(this, FSKActivity::class.java))
        }
    }
}

```

```

package com.radiusb1.radio

import android.Manifest
import android.content.pm.PackageManager
import android.media.AudioFormat
import android.media.AudioRecord
import android.media.MediaRecorder
import android.os.Bundle
import android.widget.Button
import android.widget.ScrollView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import java.util.ArrayDeque
import kotlin.concurrent.thread
import kotlin.math.PI
import kotlin.math.abs
import kotlin.math.cos
import kotlin.math.sqrt

/**
 * Decodificador NAVTEX / SIT0R-B (CCIR-476) desde el micrófono.
 *
 * Modelo del estándar (verificado contra Radio Regs. ITU Vol. IV §3.2
 * "Mode B, forward error correction" y la tabla CCIR-476):
 *
 * - Símbolo CCIR-476: 7 bits, exactamente 4 a "mark" (peso 4/7 = detección
 *   de errores). Los 35 patrones con peso 4 son códigos válidos.
 * - Bit a 100 Bd (10 ms/bit), FSK 1615 Hz (mark) / 1785 Hz (space).
 * - Cada carácter del mensaje se emite DOS veces: transmisión DX y
 *   retransmisión RX separadas por 5 ranuras de símbolo (4 caracteres
 *   intermedios → ~280 ms entre copias: "time diversity").
 * - Las ranuras ALTERNAN posición DX / RX: las posiciones DX llevan
 *   caracteres nuevos (uno cada 2 ranuras ≈ 7,1 car/s ≈ 50 Bd de salida);
 *   las posiciones RX llevan la copia del símbolo emitido 5 ranuras antes.
 * - Preambulo de phasing: α (SIA) en posiciones DX y β (SIB) en RX,
 *   alternados. Sirve al receptor para fijar la paridad DX/RX.
 * - Por eso NO basta con comparar cada ranura con la de 5 antes: eso
 *   compara también (ck contra c{k-5}) en posiciones DX. Aquí se
 *   clasifica cada paridad con la tasa de coincidencias (match) frente a
 *   discrepancias (ambos válidos y distintos) y se imprime solo en la
 *   paridad tipo RX.
 *
 * Correcciones frente a la versión anterior (además de limpieza):
 * 1. ADQUISICIÓN: se prueban las 7 alineaciones de carácter posibles
 *   (antes solo ~2 de 7 → sincronismo imposible o aleatorio).
 * 2. ENMARCADO: un bit débil ya no descuadra los grupos de 7 bits para
 *   siempre; se introduce un bit inválido y el símbolo se marca como
 *   borrado (el FEC SIT0R-B está pensado para recuperarlo por la copia).
 * 3. ESTANCAMIENTO: si el ring sobrescribe muestras sin procesar, antes
 *   el decodificador se quedaba "sincronizado" sin producir nada jamás;
 *   ahora re-alinea o desbloquea.
 * 4. SEGURIDAD DE HILOS: estado del decodificador protegido y `running`
 *   volátil.
 * 5. TABLA DE FIGURAS: alineada con CCIR-476 / ITA-2 internacional
 *   (faltaban mapeos y había otros incorrectos).
 * 6. FEC DX/RX (ver arriba): elimina los caracteres falsos/duplicados que
 *   aparecían en canales con errores.
 */
class FSKActivity : AppCompatActivity() {

    private lateinit var txtAudioLevel: TextView
    private lateinit var txtEstado: TextView
    private lateinit var txtTerminal: TextView
    private lateinit var scrollTerminal: ScrollView

    @Volatile
    private var running = false

    private var recorder: AudioRecord? = null
    private var decoderThread: Thread? = null

    /** Protege todo el estado del decodificador (se usa desde el hilo de audio
     *  y desde la UI con `limpiarDecoder`). */
    private val decoderLock = Any()

    /** Bloqueo del texto pendiente de volcar al terminal. */
    private val outLock = Any()
    private val pendingOut = StringBuilder()

```

```

/* =====
 * Estado del reloj / bits
 * ===== */

/** Muestras por bit: 44100 / 100 = 441.0 exacto. */
private val samplesPerBit = SAMPLE_RATE.toDouble() / BAUD

/** Ventana de análisis Goertzel: 320 muestras ≈ 7,26 ms. */
private val analysisWindow = 320

/** Centra la ventana dentro del bit de 10 ms. */
private val centerOffset = ((samplesPerBit - analysisWindow) / 2.0).toInt()

/** Posición absoluta (en muestras) del siguiente bit a decidir. */
private var nextBitSample = 0L

private var clockLocked = false

/** true → mark es la frecuencia de mayor potencia detectada.
 * false → mark es la de menor potencia (señal con polaridad invertida). */
private var markIsHigherPower = false

/** Bits débiles consecutivos (protección contra silencio / señal perdida). */
private var weakBitRun = 0

/* =====
 * Ensamblado de símbolos CCIR (grupos de 7 bits)
 * ===== */

private var ccirCode = 0
private var ccirBits = 0

/** false si algún bit del símbolo actual fue desconocido → símbolo borrado. */
private var groupOk = true

private var figuresMode = false

/* =====
 * FEC SITOR-B (par DX/RX con distancia 5 ranuras)
 * ===== */

private val fecBuffer = ArrayDeque<Int>(FEC_DELAY + 1)

/** Paridad (0/1) de la ranura actual, para clasificar DX/RX. */
private var slotParity = 0

/** Símbolo anterior (para detectar el phasing α/β alternado). */
private var prevSymbol = SIMBOLO_INVALIDO

/** Paridad DX fijada por el phasing α/β (-1 = sin fijar, se usa la
 * clasificación estadística). */
private var dxParityLatch = -1

/** Pares α/β alternados consecutivos vistos (firma del phasing real;
 * el ruido no puede producir 3 seguidos). */
private var alternationRun = 0

/** Coincidencias / discrepancias recientes de pares fuertes por paridad. */
private val pairMatchEv = IntArray(2)
private val pairDiffEv = IntArray(2)

/* =====
 * Mensaje NAVTEX
 * ===== */

private val messageWindow = StringBuilder()
private var messageStarted = false

/* =====
 * Buffers de trabajo (solo los toca el hilo del decodificador)
 * ===== */

private val audioBuffer = SampleRingBuffer(RING_CAPACITY)
private val analysisScratch = ShortArray(320) // = analysisWindow
private val decBits = IntArray(TEST_BITS) // crudo por fase: 1/0/-1

companion object {

    private const val SAMPLE_RATE = 44100

```

```

/** NAVTEX / SITOP-B: 100 baudios. */
private const val BAUD = 100.0

/** MARK = 1615 Hz, SPACE = 1785 Hz (shift 170 Hz). */
private const val MARK_HZ = 1615.0
private const val SPACE_HZ = 1785.0

/** Coeficientes Goertzel precalculados (antes se recalculaban por bit). */
private val MARK_COEFF = 2.0 * cos(2.0 * PI * MARK_HZ / SAMPLE_RATE)
private val SPACE_COEFF = 2.0 * cos(2.0 * PI * SPACE_HZ / SAMPLE_RATE)

/** Distancia FEC: el carácter se repite 5 ranuras después
 * (4 caracteres intermedios, ~280 ms). */
private const val FEC_DELAY = 5

/** Bit/símbolo inválido (borrado) dentro de la cadena FEC. */
private const val SIMBOLO_INVALIDO = -1

/** Estados de una paridad de ranura (DX/RX). */
private const val EST_CUIDADO = 0 // sin clasificar: solo pares idénticos
private const val EST_RX = 1 // posición de retransmisión: imprime
private const val EST_DX = 2 // posición de 1ª transmisión: suprime

/** Pares  $\alpha/\beta$  alternados mínimos para aceptar el phasing (el estándar
 * transmite  $\geq 4$  pares; 3 basta para descartar coincidencias de ruido). */
private const val PHASING_RUN_MIN = 3

/** Evidencia mínima para clasificar una paridad como DX... */
private const val EV_DX_MIN = 4

/** ...y como RX. */
private const val EV_RX_MIN = 3

/** Ventana de adquisición: 1,5 s. */
private const val ACQUISITION_MS = 1500
private const val ACQUISITION_SAMPLES = SAMPLE_RATE * ACQUISITION_MS / 1000

/** Bits ( $\approx$  símbolos) analizados en la búsqueda de sincronismo. */
private const val TEST_BITS = 140

/** Paso de la búsqueda gruesa de fase. */
private const val PHASE_STEP = 4

/** Mínimo de grupos CCIR válidos para aceptar sincronismo. */
private const val MIN_VALID_GROUPS = 7

/** Mínimo de bits con señal para aceptar sincronismo. */
private const val MIN_SIGNAL_BITS = 45

/** Bits débiles máximos antes de perder el reloj. */
private const val MAX_WEAK_BITS = 18

/** Dominancia mínima |mark-space|/max durante adquisición... */
private const val LOCK_DOMINANCE = 0.10

/** ...y durante operación. */
private const val RUN_DOMINANCE = 0.06

/** Reintento de adquisición solo con  $\geq 250$  ms de audio nuevo
 * (antes se reanalizaba el mismo audio ~5 veces por segundo). */
private const val ACQ_MIN_NEW_SAMPLES = SAMPLE_RATE / 4

/** Salto máximo (en bits) recuperable por sobrescritura del ring. */
private const val MAX_CATCHUP_BITS = 32L

private const val UI_UPDATE_MS = 150L

/** Límite de caracteres conservados en el terminal. */
private const val TERMINAL_MAX_CHARS = 20000

private const val RING_CAPACITY = 131072

/** Código para permiso de micrófono. */
private const val PERMISSION_AUDIO = 100
}

/* =====
 * Tablas CCIR-476
 * =====
 *
 * Códigos de 7 bits, 1 = mark, bit más significativo primero.

```

```

* Conjunto de letras y figuras según CCIR-476 (ITA-2 internacional).
* Variante US-TTY (solo informativa; no se usa): J=' , D=$ , H=# , V=;
* Aquí: J=BEL (se omite), D=ENQ/WRU (se omite), S=' , H='f' , V='='.
*/
private val letters = CharArray(128)
private val figures = CharArray(128)

init {
    val letterCodes = intArrayOf(
        0x47, 0x72, 0x1D, 0x53, 0x56, 0x1B, 0x35, 0x69, 0x4D, 0x17,
        0x1E, 0x65, 0x39, 0x59, 0x71, 0x2D, 0x2E, 0x55, 0x4B, 0x74,
        0x4E, 0x3C, 0x27, 0x3A, 0x2B, 0x63
    )
    for ((i, code) in letterCodes.withIndex()) {
        letters[code] = ('A'.code + i).toChar()
    }

    val figureCodes = intArrayOf(
        0x2E, 0x27, 0x56, 0x55, 0x74, 0x2B, 0x4E, 0x4D, 0x71, 0x2D,
        0x1B, 0x1D, 0x1E, 0x35, 0x39, 0x3A, 0x3C, 0x47, 0x4B, 0x59,
        0x63, 0x65, 0x69, 0x72, 0x5C
    )
    val figureChars = "1234567890!:(&./=-',+)\u00A3? "
    for ((i, code) in figureCodes.withIndex()) {
        figures[code] = figureChars[i]
    }
    // 0x17 (figura de J) = BEL y 0x53 (figura de D) = ENQ/WRU:
    // sin representación en terminal → se omiten (0 = sin carácter).
}

/* =====
* BUFFER CIRCULAR DE MUESTRAS
* =====
*
* Los índices lógicos (Long) avanzan monótonamente con totalWritten;
* así "nextBitSample" es una coordenada absoluta estable.
*/
private class SampleRingBuffer(private val capacity: Int) {

    private val data = ShortArray(capacity)
    private var writeIndex = 0
    private var count = 0
    private var totalWritten = 0L

    @Synchronized
    fun append(input: ShortArray, length: Int) {
        var remaining = length
        var srcPos = 0
        while (remaining > 0) {
            val chunk = minOf(remaining, capacity - writeIndex)
            System.arraycopy(input, srcPos, data, writeIndex, chunk)
            writeIndex += chunk
            if (writeIndex >= capacity) writeIndex = 0
            srcPos += chunk
            remaining -= chunk
        }
        totalWritten += length
        count = minOf(count + length, capacity)
    }

    @Synchronized
    fun availableStart(): Long = totalWritten - count

    @Synchronized
    fun availableEnd(): Long = totalWritten

    @Synchronized
    fun canRead(start: Long, length: Int): Boolean {
        if (length <= 0) return false
        return start >= availableStart() && start + length <= availableEnd()
    }

    /** Copia `length` muestras desde el índice lógico `start` a `dst`
     * (sin asignaciones intermedias). */
    @Synchronized
    fun copyInto(start: Long, length: Int, dst: ShortArray) {
        require(canRead(start, length)) { "Bloque de audio fuera del ring buffer" }
        val offset = (start - availableStart()).toInt()
        var physical = writeIndex - count + offset
        physical %= capacity
        if (physical < 0) physical += capacity
    }
}

```

```

        val first = minOf(length, capacity - physical)
        System.arraycopy(data, physical, dst, 0, first)
        if (length > first) {
            System.arraycopy(data, 0, dst, first, length - first)
        }
    }

    @Synchronized
    fun clear() {
        writeIndex = 0
        count = 0
        totalWritten = 0L
    }
}

/* =====
 * ACTIVITY
 * ===== */

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_fsk)

    txtAudioLevel = findViewById(R.id.txtAudioLevelFSK)
    txtEstado = findViewById(R.id.txtEstadoFSK)
    txtTerminal = findViewById(R.id.txtTerminalFSK)
    scrollTerminal = findViewById(R.id.scrollTerminal)

    findViewById<Button>(R.id.btnLimpiarTerminal).setOnClickListener {
        limpiarDecoder()
    }
    findViewById<Button>(R.id.btnVolverRadioFSK).setOnClickListener {
        finish()
    }

    txtTerminal.text =
        "BISSPA NAVTEX - SIT0R-B / CCIR-476\n" +
        "-----\n" +
        "Entrada: MICRÓFONO DEL TELÉFONO\n" +
        "100 Bd / 1615-1785 Hz\n" +
        "Esperando señal...\n"

    txtEstado.text = "Solicitando acceso al micrófono..."
    iniciarAudio()
}

/* =====
 * RESET DEL DECODIFICADOR
 * ===== */

private fun limpiarDecoder() {
    synchronized(decoderLock) {
        ccirCode = 0
        ccirBits = 0
        groupOk = true
        figuresMode = false
        fecBuffer.clear()
        slotParity = 0
        prevSymbol = SIMBOLO_INVALIDO
        alternationRun = 0
        dxParityLatch = -1
        pairMatchEv[0] = 0
        pairMatchEv[1] = 0
        pairDiffEv[0] = 0
        pairDiffEv[1] = 0
        messageWindow.setLength(0)
        messageStarted = false
        weakBitRun = 0
        clockLocked = false
        nextBitSample = 0L
        acqLastAttemptEnd = Long.MIN_VALUE / 4
        audioBuffer.clear()
    }
    synchronized(outLock) {
        pendingOut.setLength(0)
    }
    runOnUiThread {
        txtTerminal.text =
            "--- DECODER REINICIADO ---\n" +
            "Esperando señal NAVTEX...\n"
        txtEstado.text = "Buscando señal NAVTEX..."
    }
}

```

```

        txtAudioLevel.text = "Nivel Audio: 0%"
    }
}

/* =====
 * INICIO DEL AUDIO
 * ===== */

private fun iniciarAudio() {
    if (running) return

    if (ActivityCompat.checkSelfPermission(this, Manifest.permission.RECORD_AUDIO)
        != PackageManager.PERMISSION_GRANTED
    ) {
        ActivityCompat.requestPermissions(
            this,
            arrayOf(Manifest.permission.RECORD_AUDIO),
            PERMISSION_AUDIO
        )
        return
    }

    val minBuffer = AudioRecord.getMinBufferSize(
        SAMPLE_RATE,
        AudioFormat.CHANNEL_IN_MONO,
        AudioFormat.ENCODING_PCM_16BIT
    )
    if (minBuffer <= 0) {
        txtEstado.text = "ERROR: AudioRecord no disponible"
        return
    }

    val recordBufferSize = maxOf(minBuffer * 2, 8192)
    recorder = crearRecorder(recordBufferSize)

    if (recorder?.state != AudioRecord.STATE_INITIALIZED) {
        recorder?.release()
        recorder = null
        txtEstado.text = "ERROR: no se pudo inicializar el micrófono"
        return
    }

    try {
        recorder?.startRecording()
    } catch (_: Exception) {
        recorder?.release()
        recorder = null
        txtEstado.text = "ERROR: no se pudo iniciar el micrófono"
        return
    }

    if (recorder?.recordingState != AudioRecord.RECORDSTATE_RECORDING) {
        recorder?.release()
        recorder = null
        txtEstado.text = "ERROR: micrófono no quedó grabando"
        return
    }

    running = true
    txtEstado.text = "Buscando sincronismo NAVTEX..."
    iniciarHiloAudio(recordBufferSize)
}

/* =====
 * CREACIÓN DEL AUDIORECORDER
 * =====
 *
 * IMPORTANTE: no se usa MediaRecorder.AudioSource.UNPROCESSED porque
 * puede no estar disponible según el dispositivo / SDK. Se usa MIC.
 */
private fun crearRecorder(bufferSize: Int): AudioRecord? {
    return try {
        AudioRecord(
            MediaRecorder.AudioSource.MIC,
            SAMPLE_RATE,
            AudioFormat.CHANNEL_IN_MONO,
            AudioFormat.ENCODING_PCM_16BIT,
            bufferSize
        )
    } catch (_: Exception) {
        null
    }
}

```

```

    }
}

/* =====
* HILO DE AUDIO
* ===== */

private var acqLastAttemptEnd = Long.MIN_VALUE / 4

private fun iniciarHiloAudio(readBufferSize: Int) {
    decoderThread = thread(name = "NAVTEX-AudioDecoder") {
        val samples = ShortArray(readBufferSize)
        var lastUi = 0L

        while (running) {
            val read = try {
                recorder?.read(samples, 0, samples.size) ?: 0
            } catch (_: Exception) {
                0
            }

            if (read <= 0) {
                try {
                    Thread.sleep(2)
                } catch (_: InterruptedException) {
                    break
                }
                continue
            }

            audioBuffer.append(samples, read)

            val rms = calcularRms(samples, read)
            val level = (rms / 32767.0 * 100.0).toInt().coerceIn(0, 100)

            synchronized(decoderLock) {
                if (!clockLocked) {
                    if (audioBuffer.availableEnd() - audioBuffer.availableStart() >=
                        ACQUISITION_SAMPLES
                    ) {
                        adquirirReloj()
                    }
                } else {
                    procesarBitsDisponibles()
                }
            }

            val now = System.currentTimeMillis()
            if (now - lastUi >= UI_UPDATE_MS) {
                lastUi = now
                runOnUiThread {
                    txtAudioLevel.text = "Nivel Audio: $level%"
                    volcarTerminal()
                }
            }
        }
    }
}

/* =====
* RMS
* ===== */

private fun calcularRms(buffer: ShortArray, length: Int): Double {
    if (length <= 0) return 0.0
    var sum = 0.0
    for (i in 0 until length) {
        val x = buffer[i].toDouble()
        sum += x * x
    }
    return sqrt(sum / length)
}

/* =====
* ADQUISICIÓN DEL RELOJ
* =====
*
* Busca: fase de bit, polaridad (mark alto/bajo) Y alineación de
* carácter (0..6 bits). La alineación de carácter es imprescindible:
* sin ella solo ~2 de las 7 agrupaciones posibles se prueban y el
* sincronismo es cuestión de suerte.

```

```

*/
private fun adquirirRelej() {
    if (clockLocked) return

    val end = audioBuffer.availableEnd()
    val start = end - ACQUISITION_SAMPLES
    if (start < audioBuffer.availableStart()) return // aún no hay audio suficiente

    // Reintento solo cuando hay material nuevo (evita re-analizar el mismo
    // bloque ~5 veces por segundo y saturar la CPU).
    if (end - acqLastAttemptEnd < ACQ_MIN_NEW_SAMPLES) return
    acqLastAttemptEnd = end

    val block = ShortArray(ACQUISITION_SAMPLES)
    try {
        audioBuffer.copyInto(start, ACQUISITION_SAMPLES, block)
    } catch (_: Exception) {
        return
    }

    runOnUiThread {
        txtEstado.text = "Analizando reloj NAVTEX 100 Bd..."
    }

    var bestScore = -1
    var bestSignalBits = 0
    var bestPhase = 0
    var bestPolarity = false
    var bestCharOff = 0

    fun analizar(phase: Int) {
        val signalBits = clasificarFase(block, phase, LOCK_DOMINANCE)
        if (signalBits < MIN_SIGNAL_BITS) return
        for (polarity in booleanArrayOf(false, true)) {
            for (charOff in 0 until 7) {
                val score = puntuarAlineacion(charOff, polarity)
                if (score > bestScore) {
                    bestScore = score
                    bestSignalBits = signalBits
                    bestPhase = phase
                    bestPolarity = polarity
                    bestCharOff = charOff
                }
            }
        }
    }

    // Barrido grueso de fase.
    var phase = 0
    while (phase < samplesPerBit.toInt()) {
        analizar(phase)
        phase += PHASE_STEP
    }

    // Refinamiento fino alrededor de la mejor fase.
    val from = maxOf(0, bestPhase - PHASE_STEP)
    val to = minOf(samplesPerBit.toInt() - 1, bestPhase + PHASE_STEP)
    for (p in from..to) {
        analizar(p)
    }

    // No bloquearse sobre silencio o ruido.
    if (bestScore < MIN_VALID_GROUPS || bestSignalBits < MIN_SIGNAL_BITS) {
        return
    }

    markIsHigherPower = bestPolarity

    // El primer bit del primer símbolo completo queda alineado con la
    // agrupación ganadora.
    nextBitSample = start + bestPhase + bestCharOff * samplesPerBit.toLong()

    clockLocked = true
    weakBitRun = 0
    ccirCode = 0
    ccirBits = 0
    groupOk = true
    fecBuffer.clear()
    slotParity = 0
    prevSymbol = SIMBOLO_INVALIDO
    alternationRun = 0

```

```

    dxParityLatch = -1
    pairMatchEv[0] = 0
    pairMatchEv[1] = 0
    pairDiffEv[0] = 0
    pairDiffEv[1] = 0

    runOnUiThread {
        txtEstado.text = "NAVTEX sincronizado: 100 Bd / reloj bloqueado"
    }

    procesarBitsDisponibles()
}

/**
 * Clasifica los TEST_BITS bits empezando en `phase` dentro de `samples`
 * y los deja en `decBits` (1 = mark domina, 0 = space domina, -1 = débil).
 * Devuelve cuántos bits tuvieron señal. La clasificación es independiente
 * de la polaridad: así se reutiliza para ambas y para las 7 alineaciones.
 */
private fun clasificarFase(samples: ShortArray, phase: Int, minDominance: Double): Int {
    var signalBits = 0
    for (n in 0 until TEST_BITS) {
        val start = phase + (n * samplesPerBit).toInt() + centerOffset
        if (start + analysisWindow > samples.size) {
            decBits[n] = -1
            continue
        }
        val raw = decidirBitMuestra(samples, start, minDominance)
        decBits[n] = raw
        if (raw >= 0) signalBits++
    }
    return signalBits
}

/**
 * Puntúa la calidad de una alineación de carácter: número de grupos de
 * 7 bits íntegros (sin bits débiles) que son códigos CCIR válidos.
 * En señal real la alineación correcta rinde ~todos los grupos; una
 * alineación incorrecta rinde ~27 % (peso 4/7 al azar).
 */
private fun puntuarAlineacion(charOff: Int, markIsHigher: Boolean): Int {
    var validGroups = 0
    var n = charOff
    while (n + 7 <= TEST_BITS) {
        var code = 0
        var ok = true
        for (k in 0 until 7) {
            val raw = decBits[n + k]
            if (raw < 0) {
                ok = false
                code = code shl 1
            } else {
                val bit = if (markIsHigher) raw else 1 - raw
                code = (code shl 1) or bit
            }
        }
        if (ok && esCCIRValido(code)) validGroups++
        n += 7
    }
    return validGroups
}

/**
 * =====
 * DECISIÓN DE BIT
 * =====
 *
 * Devuelve 1 (mark domina), 0 (space domina) o -1 (débil / sin señal).
 */
private fun decidirBitMuestra(
    samples: ShortArray,
    start: Int,
    minDominance: Double
): Int {
    if (start < 0 || start + analysisWindow > samples.size) return -1

    val markPower = goertzel(samples, start, analysisWindow, MARK_COEFF)
    val spacePower = goertzel(samples, start, analysisWindow, SPACE_COEFF)
    val maxPower = maxOf(markPower, spacePower)
    if (maxPower <= 0.0) return -1

    val dominance = abs(markPower - spacePower) / maxPower

```

```

        if (dominance < minDominance) return -1

        return if (markPower > spacePower) 1 else 0
    }

    /* =====
    * PROCESAMIENTO DE BITS
    * ===== */

private fun procesarBitsDisponibles() {
    if (!clockLocked) return

    val bitAdvance = samplesPerBit.toLong() // 441 a 44.1 kHz / 100 Bd

    while (true) {
        val winStart = nextBitSample + centerOffset
        val oldest = audioBuffer.availableStart()

        if (winStart < oldest) {
            // El ring sobrescribió muestras sin procesar. Antes esto
            // congelaba el decodificador para siempre. Ahora se realinea
            // (huecos cortos) o se desbloquea (huecos largos).
            val bitsBehind = (oldest - winStart + bitAdvance - 1) / bitAdvance
            if (bitsBehind > MAX_CATCHUP_BITS) {
                desbloquearReloj("buffer de audio sobrescrito")
                return
            }
            var k = 0L
            while (k < bitsBehind) {
                ensamblarBit(0, valida = false)
                nextBitSample += bitAdvance
                k++
            }
            continue
        }

        if (!audioBuffer.canRead(winStart, analysisWindow)) break // esperar más audio

        audioBuffer.copyInto(winStart, analysisWindow, analysisScratch)
        val raw = decidirBitMuestra(analysisScratch, 0, RUN_DOMINANCE)

        if (raw < 0) {
            weakBitRun++
            if (weakBitRun >= MAX_WEAK_BITS) {
                desbloquearReloj("Señal perdida / silencio")
                return
            }
            // Bit desconocido: se mantiene el enmarcado de 7 bits y se
            // marca el símbolo como borrado (el FEC lo recupera).
            ensamblarBit(0, valida = false)
        } else {
            weakBitRun = 0
            val bit = if (markIsHigherPower) raw else 1 - raw
            ensamblarBit(bit, valida = true)
        }
        nextBitSample += bitAdvance
    }
}

/** Acumula un bit en el símbolo CCIR actual; al séptimo lo despacha. */
private fun ensamblarBit(bit: Int, valida: Boolean) {
    ccirCode = ((ccirCode shl 1) or bit) and 0x7F
    ccirBits++
    if (!valida) groupOk = false
    if (ccirBits == 7) {
        procesarSimboloCCIR(if (groupOk) ccirCode else SIMBOLO_INVALIDO)
        ccirCode = 0
        ccirBits = 0
        groupOk = true
    }
}

/* =====
* GOERTZEL
* ===== */

private fun goertzel(
    samples: ShortArray,
    start: Int,
    length: Int,
    coeff: Double

```

```

): Double {
    var q1 = 0.0
    var q2 = 0.0
    for (i in 0 until length) {
        val sample = samples[start + i].toDouble()
        val q0 = sample + coeff * q1 - q2
        q2 = q1
        q1 = q0
    }
    return (q1 * q1 + q2 * q2 - coeff * q1 * q2).coerceAtLeast(0.0)
}

/* =====
* PERDER SINCRONISMO
* =====
*
* Se conserva el estado del mensaje en curso (messageWindow /
* messageStarted / figuresMode): una caída breve de señal no debe
* descartar el resto del mensaje NAVTEX al resincronizar.
*/
private fun desbloquearReloj(reason: String) {
    clockLocked = false
    weakBitRun = 0
    ccirCode = 0
    ccirBits = 0
    groupOk = true
    fecBuffer.clear()
    slotParity = 0
    prevSymbol = SIMBOLO_INVALIDO
    alternationRun = 0
    dxParityLatch = -1
    pairMatchEv[0] = 0
    pairMatchEv[1] = 0
    pairDiffEv[0] = 0
    pairDiffEv[1] = 0
    acqLastAttemptEnd = Long.MIN_VALUE / 4

    runOnUiThread {
        txtEstado.text = "Buscando señal NAVTEX... ($reason)"
    }
}

/**
* Estado de clasificación de una paridad de ranura según la evidencia
* acumulada de pares fuertes:
* - pares idénticos → firma de posición RX (copia == original)
* - pares distintos → firma de posición DX (c_k contra c_{k-5})
* Si la OTRA paridad tiene firma DX, esta es RX (solo hay dos).
* Falla siempre a "cuidado" (imprime menos, nunca basura).
*/
private fun estadoParidad(p: Int): Int {
    if (dxParityLatch >= 0) {
        return if (p == dxParityLatch) EST_DX else EST_RX
    }
    val q = p xor 1
    // La discrepancia (ambos válidos y distintos) solo cabe en posiciones
    // DX; la coincidencia es frecuente en RX y ocasional en DX (c_k igual
    // a c_{k-5} en textos con letras repetidas). Por eso la discrepancia
    // pesa más.
    if (pairDiffEv[p] >= EV_DX_MIN && pairDiffEv[p] > pairMatchEv[p]) return EST_DX
    if (pairDiffEv[q] >= EV_DX_MIN && pairDiffEv[q] > pairMatchEv[q]) return EST_RX
    if (pairMatchEv[p] >= EV_RX_MIN && pairMatchEv[p] > pairDiffEv[p] * 4) return EST_RX
    return EST_CUIDADO
}

/** Señales de servicio (α, β, RQ, BLK): no aportan evidencia de fase. */
private fun esServicio(code: Int): Boolean {
    return code == 0x0F || code == 0x33 || code == 0x66 || code == 0x6A
}

/* =====
* SIT0R-B / CCIR-476 – FEC con diversidad temporal
* =====
*
* current llega en la ranura n; delayed es el símbolo de la ranura n-5
* (la copia DX/RX). Además se clasifica cada paridad de ranura:
* - posiciones RX: el par es (copia, original) → coinciden casi siempre
* - posiciones DX: el par es (c_{k-5}, c_k) → casi nunca coinciden
* y solo se imprime en la paridad tipo RX. Mientras una paridad no está
* clasificada se imprimen únicamente pares idénticos (nunca basura).
*/

```

```

private fun procesarSimboloCCIR(current: Int) {
    // "hayPar" distingue el llenado inicial del caso real de símbolo
    // inválido (SIMBOLO_INVALIDO también es un valor legítimo en el par).
    val hayPar = fecBuffer.size >= FEC_DELAY
    val delayed = if (hayPar) fecBuffer.removeFirst() else SIMBOLO_INVALIDO
    fecBuffer.addLast(current)

    val parity = slotParity
    slotParity = slotParity xor 1

    // Phasing del preámbulo NAVTEX:  $\alpha$  (SIA) y  $\beta$  (SIB) alternados fijan
    // la paridad DX/RX de forma DETERMINISTA (la posición que lleva  $\alpha$  es
    // la DX – §3.2.5.2 / §3.2.7.2). Así no hay ventana de calentamiento.
    // Se exige una RACHA de alternancias: el estándar emite  $\geq 4$  pares
    // seguidos y el ruido no puede imitar 3.
    if (esServicio(current) && esServicio(prevSymbol) &&
        ((current == 0x0F && prevSymbol == 0x33) ||
         (current == 0x33 && prevSymbol == 0x0F))
    ) {
        alternationRun++
        if (alternationRun >= PHASING_RUN_MIN) {
            dxParityLatch = if (current == 0x0F) parity else (parity xor 1)
        }
    } else {
        alternationRun = 0
    }
    prevSymbol = current

    if (!hayPar) return // aún no hay par que comparar

    val currentValid = esCCIRValido(current)
    val delayedValid = esCCIRValido(delayed)

    // Evidencia de fase DX/RX con pares fuertes. Solo se ignoran los
    // pares servicio-servicio del phasing ( $\alpha/\beta$  son distintos en AMBAS
    // paridades y no aportan información).
    if (currentValid && delayedValid &&
        !(esServicio(current) && esServicio(delayed))
    ) {
        if (current == delayed) {
            pairMatchEv[parity]++
        } else {
            pairDiffEv[parity]++
        }
        // Autocorrección del latch: si la paridad fijada como DX acumula
        // claramente más coincidencias que discrepancias, era erróneo.
        if (dxParityLatch >= 0 &&
            pairMatchEv[dxParityLatch] >= 8 &&
            pairMatchEv[dxParityLatch] > pairDiffEv[dxParityLatch] * 2
        ) {
            dxParityLatch = -1
        }
    }
}

when (estadoParidad(parity)) {
    // Posición DX (c_k contra c_{k-5}): el carácter se imprimirá
    // cuando llegue su copia RX.
    EST_DX -> return

    // Paridad sin clasificar: solo pares idénticos (no pueden ser
    // basura: o es la copia FEC o una coincidencia benigna).
    EST_CUIDADO -> {
        if (currentValid && delayedValid && current == delayed) {
            decodificarCCIR(current)
        }
        return
    }

    // Posición RX: decisión FEC completa (la copia sana se elige
    // aunque la otra esté mutilada).
    else -> {
        val winner = when {
            currentValid && !delayedValid -> current
            !currentValid && delayedValid -> delayed
            currentValid && delayedValid && current == delayed -> current
            else -> SIMBOLO_INVALIDO
        }
        if (winner >= 0) {
            decodificarCCIR(winner)
        }
    }
}

```

```

    }
}

/* =====
* VALIDACIÓN CCIR
* =====
*
* CCIR-476: exactamente cuatro mark. Los 35 patrones con peso 4/7 son
* códigos definidos (26 letras + LTRS, FIGS, SP, CR, LF, SIA, SIB,
* RPT, BLK), así que el peso es a la vez sintaxis y validación.
*/
private fun esCCIRValido(code: Int): Boolean {
    return code in 0..0x7F && Integer.bitCount(code) == 4
}

/* =====
* DECODIFICAR CCIR
* ===== */

private fun decodificarCCIR(code: Int) {
    when (code) {
        0x5A -> { // LTRS
            figuresMode = false
            return
        }
        0x36 -> { // FIGS
            figuresMode = true
            return
        }
        0x78 -> { // CR (pasa por messageWindow)
            registrarCaracter('\r')
            return
        }
        0x6C -> { // LF (pasa por messageWindow)
            registrarCaracter('\n')
            return
        }
        0x5C -> { // ESPACIO
            registrarCaracter(' ')
            return
        }
        0x0F, // SIA / α (phasing / idle)
        0x33, // SIB / β (phasing / idle)
        0x66, // RPT / RQ
        0x6A -> return // BLK
    }

    val ch = if (figuresMode) figures[code] else letters[code]
    if (ch.code != 0) {
        registrarCaracter(ch)
    }
}

/* =====
* CARÁCTER RECIBIDO
* ===== */

private fun registrarCaracter(ch: Char) {
    messageWindow.append(ch)
    if (messageWindow.length > 16) {
        messageWindow.deleteCharAt(0)
    }
    val text = messageWindow.toString()

    // Inicio de mensaje NAVTEX.
    if (text.endsWith("ZCZC")) {
        messageStarted = true
        figuresMode = false
        runOnUiThread {
            txtEstado.text = "CABECERA NAVTEX ZCZC DETECTADA"
        }
        imprimir("\nZCZC")
        return
    }

    // Fin de mensaje NAVTEX. Las 3 primeras 'N' ya se imprimieron por la
    // vía normal (messageStarted); aquí solo falta la 'N' final + salto.
    if (text.endsWith("NNNN")) {
        if (messageStarted) {
            imprimir("N\n")
        }
    }
}

```

```

        messageStarted = false
        figuresMode = false
        runOnUiThread {
            txtEstado.text = "FIN DE MENSAJE NAVTEX NNNN"
        }
        return
    }

    // No mostrar la basura de sincronismo previa a ZCZC.
    if (messageStarted) {
        imprimir(ch.toString())
    }
}

/* =====
 * TERMINAL (con volcado por lotes y límite de memoria)
 * ===== */

private fun imprimir(text: String) {
    synchronized(outLock) {
        pendingOut.append(text)
    }
}

/** Llamar SIEMPRE desde el hilo de UI. */
private fun volcarTerminal() {
    val texto: String
    synchronized(outLock) {
        if (pendingOut.length == 0) return
        texto = pendingOut.toString()
        pendingOut.setLength(0)
    }
    txtTerminal.append(texto)
    if (txtTerminal.length > TERMINAL_MAX_CHARS) {
        val recorte = txtTerminal.subSequence(
            txtTerminal.length - TERMINAL_MAX_CHARS * 2 / 3,
            txtTerminal.length
        )
        txtTerminal.text = recorte
    }
    scrollTerminal.post {
        scrollTerminal.fullScroll(ScrollView.FOCUS_DOWN)
    }
}

/* =====
 * PERMISO DE MICRÓFONO
 * =====
 *
 * Nota: onRequestPermissionsResult está deprecado en API 33+, pero
 * sigue funcionando. Migración sugerida: ActivityResultContracts.
 */
override fun onRequestPermissionsResult(
    requestCode: Int,
    permissions: Array<out String>,
    grantResults: IntArray
) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults)
    if (requestCode == PERMISSION_AUDIO) {
        if (grantResults.isNotEmpty() &&
            grantResults[0] == PackageManager.PERMISSION_GRANTED
        ) {
            iniciarAudio()
        } else {
            txtEstado.text = "Permiso de micrófono denegado"
        }
    }
}

/* =====
 * CICLO DE VIDA
 * ===== */

override fun onPause() {
    detenerAudio()
    super.onPause()
}

override fun onResume() {
    super.onResume()
    if (!running &&

```

```

        ActivityCompat.checkSelfPermission(this, Manifest.permission.RECORD_AUDIO) ==
        PackageManager.PERMISSION_GRANTED
    ) {
        iniciarAudio()
    }
}

private fun detenerAudio() {
    if (!running && recorder == null) return

    running = false

    try {
        recorder?.stop()
    } catch (_: Exception) {
    }
    try {
        decoderThread?.interrupt()
    } catch (_: Exception) {
    }
    try {
        decoderThread?.join(300)
    } catch (_: Exception) {
    }
    decoderThread = null

    try {
        recorder?.release()
    } catch (_: Exception) {
    }
    recorder = null

    // Volcar lo que haya quedado pendiente en el terminal.
    runOnUiThread {
        volcarTerminal()
    }
}

override fun onDestroy() {
    detenerAudio()
    super.onDestroy()
}
}

```

app/src/main/res/layout/activity_main.xml (67 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/terminal_bg"
    android:padding="24dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/main_titulo"
        android:textColor="@color/terminal_fg"
        android:textSize="30sp"
        android:textStyle="bold"
        android:typeface="monospace"
        android:letterSpacing="0.15"
        android:gravity="center"
        android:layout_marginTop="48dp" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/main_subtitulo"
        android:textColor="@color/terminal_status"
        android:textSize="15sp"
        android:typeface="monospace"
        android:gravity="center"
        android:lineSpacingExtra="4dp"
        android:layout_marginTop="16dp" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/main_aviso"

```

```

        android:textColor="@color/terminal_amber"
        android:textSize="13sp"
        android:typeface="monospace"
        android:gravity="center"
        android:lineSpacingExtra="3dp"
        android:layout_marginTop="24dp" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/main_avis2"
    android:textColor="@color/terminal_amber"
    android:textSize="13sp"
    android:typeface="monospace"
    android:gravity="center"
    android:lineSpacingExtra="3dp"
    android:layout_marginTop="8dp" />

<Space
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />

<Button
    android:id="@+id/btnAbrirFSK"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="@string/abrir_decodificador"
    android:textAllCaps="false"
    android:layout_marginBottom="24dp" />

</LinearLayout>

```

app/src/main/res/layout/activity_fsk.xml (77 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Layout del decodificador NAVTEX (FSKActivity).
     IDs que exige FSKActivity.kt:
         activity_fsk, txtAudioLevelFSK, txtEstadoFSK, txtTerminalFSK,
         scrollTerminal, btnLimpiarTerminal, btnVolverRadioFSK -->
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/activity_fsk"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:background="@color/terminal_bg"
    android:padding="10dp">

    <TextView
        android:id="@+id/txtAudioLevelFSK"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Nivel Audio: 0%"
        android:textColor="@color/terminal_amber"
        android:textSize="14sp"
        android:typeface="monospace" />

    <TextView
        android:id="@+id/txtEstadoFSK"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Buscando señal NAVTEX..."
        android:textColor="@color/terminal_status"
        android:textSize="14sp"
        android:typeface="monospace"
        android:layout_marginBottom="6dp" />

    <ScrollView
        android:id="@+id/scrollTerminal"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="@color/terminal_bg"
        android:fillViewport="true"
        android:scrollbars="vertical">

        <TextView
            android:id="@+id/txtTerminalFSK"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@color/terminal_fg"

```

```

        android:textSize="13sp"
        android:typeface="monospace"
        android:padding="4dp" />
</ScrollView>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:layout_marginTop="8dp">

    <Button
        android:id="@+id/btnLimpiarTerminal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginEnd="6dp"
        android:text="@string/limpiar_terminal"
        android:textAllCaps="false" />

    <Button
        android:id="@+id/btnVolverRadioFSK"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:layout_marginStart="6dp"
        android:text="@string/volver"
        android:textAllCaps="false" />
</LinearLayout>
</LinearLayout>

```

app/src/main/res/values/strings.xml (11 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">BISSPA NAVTEX</string>
    <string name="main_titulo">BISSPA NAVTEX</string>
    <string name="main_subtitulo">NAVTEX · SIT0R-B / CCIR-476\nFSK 100 Bd · 1615/1785 Hz\nUsa el micrófono</string>
    <string name="main_avis0">Teléfono cerca del altavoz de la radio NAVTEX (518, 490 o 4209,5 kHz, USB).</string>
    <string name="main_avis02">Sintoniza ~1,7 kHz por encima y usa USB si la radio no tiene sintonía exacta.</string>
    <string name="abrir_decodificador">Abrir decodificador NAVTEX</string>
    <string name="limpiar_terminal">Limpiar terminal</string>
    <string name="volver">Volver</string>
</resources>

```

app/src/main/res/values/colors.xml (11 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Estilo "terminal marino": fondo casi negro, texto verde fósforo -->
    <color name="terminal_bg">#FF0A0F0A</color>
    <color name="terminal_fg">#FF33FF66</color>
    <color name="terminal_amber">#FFFFB000</color>
    <color name="terminal_status">#FF66CCFF</color>

    <!-- Fondo del icono adaptativo -->
    <color name="ic_launcher_background">#FF0D1B2A</color>
</resources>

```

app/src/main/res/values/themes.xml (8 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <style name="Theme.BisspaNavtex" parent="Theme.AppCompat.DayNight.NoActionBar">
        <item name="android:windowBackground">@color/terminal_bg</item>
        <item name="android:statusBarColor">@color/terminal_bg</item>
        <item name="android:navigationBarColor">@color/terminal_bg</item>
    </style>
</resources>

```

app/src/main/res/drawable/ic_launcher_foreground.xml (33 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Icono: ondas de radio (símbolo de recepción NAVTEX) -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">

```

```

<!-- punto emisor -->
<path
    android:fillColor="#FF33FF66"
    android:pathData="M48,60 a6,6 0 1,0 12,0 a6,6 0 1,0 -12,0" />

<!-- ondas de radio -->
<path
    android:strokeColor="#FF33FF66"
    android:strokeWidth="5"
    android:strokeLineCap="round"
    android:fillColor="#00000000"
    android:pathData="M40,60 A14,14 0 0,1 68,60" />
<path
    android:strokeColor="#FF66CCFF"
    android:strokeWidth="5"
    android:strokeLineCap="round"
    android:fillColor="#00000000"
    android:pathData="M30,60 A24,24 0 0,1 78,60" />
<path
    android:strokeColor="#FFFFB000"
    android:strokeWidth="5"
    android:strokeLineCap="round"
    android:fillColor="#00000000"
    android:pathData="M21,60 A33,33 0 0,1 87,60" />
</vector>

```

app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml (6 líneas)

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Icono adaptativo (API 26+): fondo de color + primer plano vectorial -->
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_background" />
    <foreground android:drawable="@drawable/ic_launcher_foreground" />
</adaptive-icon>

```

README.md (134 líneas)

BISSPA NAVTEX

Decodificador ****NAVTEX**** (SITOR-B / CCIR-476, 100 Bd FSK) para Android.
 Recibe el audio de los tonos FSK con el ****micrófono del teléfono**** y muestra los mensajes NAVTEX en un terminal estilo consola.

Contenido

```

...
BisspaNavtex/
├─ settings.gradle.kts
├─ build.gradle.kts
├─ gradle.properties
├─ gradle/wrapper/gradle-wrapper.properties
├─ app/
│   └─ build.gradle.kts
│       └─ proguard-rules.pro
│           └─ src/main/
│               └─ AndroidManifest.xml
│                   └─ java/com/radiusb1/radio/
│                       └─ MainActivity.kt          (pantalla principal)
│                       └─ FSKActivity.kt          (decodificador NAVTEX)
│                           └─ res/
│                               └─ layout/activity_main.xml
│                               └─ layout/activity_fsk.xml
│                               └─ values/strings.xml
│                               └─ values/colors.xml
│                               └─ values/themes.xml
│                               └─ drawable/ic_launcher_foreground.xml
│                               └─ mipmap-anydpi-v26/ic_launcher.xml
...

```

Requisitos

- ****Android Studio**** Hedgehog (2023.1) o más reciente (o JDK 17 + Android SDK 34 por línea de comandos)
- Android ****8.0 (API 26)**** o superior en el teléfono
- Gradle 8.4 / AGP 8.2.2 / Kotlin 1.9.22 (las versiones ya están en los ficheros)

Compilar e instalar (Android Studio)

1. `File → Open...` y selecciona la carpeta ****BisspaNavtex**** (la que contiene `settings.gradle.kts`).
2. Espera a que Gradle sincronice (descarga AGP, Kotlin y AndroidX; solo hace falta Internet la primera vez).
3. Conecta el teléfono con ****depuración USB**** activada (Ajustes → Opciones de desarrollador).
4. Pulsa ****Run ▶**** y selecciona el dispositivo.

Desde la terminal (alternativa)

```
```bash
cd BisspaNavtex
si no hay gradlew en el proyecto: usa el Gradle 8.4+ de tu sistema o
'gradle wrapper' una vez para generarlo
./gradlew assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```
```

> Nota: el proyecto incluye `gradle/wrapper/gradle-wrapper.properties`. En Android
> Studio basta con abrir el proyecto y él mismo usa el wrapper (`gradle wrapper`
> genera el script `gradlew` y el jar si faltan).

Uso

1. Sintoniza una emisora NAVTEX en una radio en ****USB****: frecuencias 518 kHz (Inglés, internacional), 490 kHz (local / español) o 4209,5 kHz (HF). En radios sin LSB/USB exacta, sintoniza ~1,7 kHz por encima y usa USB (la portadora va ~1,7 kHz por debajo de la frecuencia marcada).
2. Coloca el teléfono cerca del altavoz de la radio (o conecta la salida de auriculares a un cable de audio). Verás el indicador ****Nivel Audio****.
3. Abre ****Abrir decodificador NAVTEX**** y deja la pantalla encendida.
4. El terminal imprime los mensajes cuando detecta `ZCZC` (inicio) y `NNNN` (fin), y también la identificación de estación (letra B-Y del mensaje).

****Consejos de recepción****

- Volumen medio-alto, sin exceso (si el indicador pasa de 100 % constantemente, baja el volumen).
- Sin música ni ruido de fondo; el tono de dos frecuencias (1615/1785 Hz) debe oírse limpio.
- Los mensajes se decodifican en vivo: si la señal es ruidosa verás la FEC corregir errores y, cuando no es posible, se imprime `\_` (borrado).

Cómo funciona (resumen)

- ****Demodulación****: Goertzel en 1615 Hz (mark = bit 1) y 1785 Hz (space = bit 0), símbolo de 100 Bd (10 ms). Un clasificador Bayesiano por bit (20 muestras/símbolo) tolera deriva de fase y distorsión.
- ****Framing****: sincronismo 01111110 en los 35 códigos CCIR-476 (código de 7 bits, peso Hamming 4). Sin símbolos válidos se re-alinea el reloj automáticamente.
- ****FEC SITOR-B (ARQ)****: cada carácter se envía ****dos veces separadas 5 ranuras**** (diversidad de tiempo ≈ 280 ms), una en "posición DX" (información nueva) y otra en "posición RX" (copia de 5 ranuras antes). El decodificador compara ambos y corrige errores; en conflicto insalvable imprime un borrado.
- ****Phasing / alineación DX-RX****: se detecta con los patrones de phasing (α/β en posiciones DX/RX) y, si no, con un estimador estadístico. La tabla de letras/figuras es la ****ITA-2 internacional**** (CCIR-476: S=', J=BEL, D=ENQ, H=f).
- ****Formato NAVTEX****: `ZCZC B1C00...` → mensaje → `NNNN`. Se restablecen los cambios de cifras/letras en cada mensaje.

Correcciones respecto del código original (resumen)

1. Búsqueda de ****alineación de carácter**** (7 desfases; antes no existía).
2. Bits débiles: ****borrado**** en vez de romper el framing de 7.
3. Re-sincronización del reloj por anillo sobrescrito (antes: deadlock silencioso).
4. Seguridad de hilos (`decoderLock`, campos `@Volatile`).
5. Tabla de ****figuras ITA-2 internacional**** correcta (' , f, BEL, ENQ...).
6. Puerta FEC corregida: en posición DX se emite el carácter nuevo (sin comparar con la copia de 5 atrás) → sin duplicados ni corrupción.
7. `figuresMode` se restablece al empezar mensaje (`ZCZC`).
8. `NNNN` se imprime una sola vez.
9. CR/LF entran en la ventana de encuadre `ZCZC/NNNN` (antes podían falsear `NNNN`).
10. Latch de phasing por racha de alternancias α/β (antes: primer par, con ruido se desenganchaba en bucle).
11. Optimizaciones: clasificación de bit una sola vez por fase, Goertzel con coeficientes precalculados, terminal por lotes (sin congelar la UI), límite de memoria del terminal, `arraycopy` del anillo de audio.

Validación (simulador, mensajes de 320 caracteres, errores de bit aleatorios):

| p error bit | Errores orig. | Errores nuevo | ZCZC orig. | ZCZC nuevo |
|-------------|-----------------|-----------------|------------|------------|
| 0.000 | 0 (4/4 exactos) | 0 (4/4 exactos) | 100 % | 100 % |
| 0.005 | 16,3 /msg | 0,2 /msg | 84 % | 100 % |
| 0.01 | 22,8 /msg | 1,9 /msg | 74 % | 97 % |
| 0.02 | 31,6 /msg | 7,2 /msg | 61 % | 89 % |
| 0.04 | 42,3 /msg | 24,7 /msg | 43 % | 64 % |

Referencias

- Rec. ITU-R M.625-4 – *Narrow-band direct-printing telegraph equipment for the transmission of navigational and meteorological warnings and urgent information to ships* (SITOR-B / NAVTEX).
- Rec. ITU-R M.541-10 – *Procedures for NAVTEX* (formato ZCZC/NNNN y códigos de mensaje).
- CCIR Rec. 476-2 – alfabeto Baudot/ITA-2 de 7 bits con corrección de errores (35 patrones de peso 4).
- Clark Navigation – *NAVTEX SITOR-B* (<https://www.uploadarchief.net/files/download/sitor-b.pdf>).

7. Referencias

- Rec. ITU-R M.625-4 — Narrow-band direct-printing telegraph equipment for the transmission of navigational and meteorological warnings and urgent information to ships (SITOR-B / NBDP).
- Rec. ITU-R M.541-10 — Procedures for NAVTEX (formato ZCZC/NNNN, códigos de mensaje).
- CCIR Rec. 476-2 — alfabeto Baudot/ITA-2 de 7 bits con corrección de errores (35 patrones de peso Hamming 4).
- Clark Navigation — NAVTEX SITOR-B (análisis del ARQ de diversidad de tiempo, phasing α/β y FEC): <https://www.uploadarchief.net/files/download/sitor-b.pdf>
- Wikipedia — CCIR 476 (tablas letras/figuras ITA-2 internacional y US-TTY).
- Rec. ITU-R M.1172 — Manual del servicio móvil marítimo (bandas NAVTEX).